Structural losses for heavy rain: the Minkowski loss, flow matching, and where the gradient goes

minkowski-loss project notes (DECISIONS §37–§40)

30 October 2026

What these notes are for. They collect, in one place and from first principles, everything needed to understand how the Minkowski loss acts on the two models of the project: the deterministic U-Net (Study 1) and the flow-matching residual model (Study 2). Every loss is written out. Every claim carries a label: [derivation] follows from the stated definitions; [measured] was run in this project (the file is given); [inference] is our reading, not tested directly; [withdrawn] was claimed earlier and is wrong. All numbers refer to the hourly v4 data (`patches_v4_adv`), the hourly vanilla backbone `UNet_Ana_20261021_211732` (last checkpoint) and the clean flow-matching model `Extremes_20261026_092630/fm_best.pth`, on validation subsets, with one training seed.

Contents

1 The problem in one page **2**

2 Fields, the two spaces, and the base models **2**
2.1 Model space . . . 2
2.2 The deterministic backbone and its loss (Study 1) . . . 2
2.3 The flow-matching residual model (Study 2) . . . 3

3 The Minkowski functionals and the structural distance **4**
3.1 Study 1 on hourly data, in brief . . . 4

4 How the Minkowski loss is coupled to flow matching **5**
4.1 The distributional reward . . . 6
4.2 The energy reward . . . 6
4.3 The proximal anchor and the retention term . . . 7

5 How the terms share the update: the gradient budget **7**
5.1 What the two fine-tunes did . . . 7

6 Why the original reward misses heavy rain **8**

7 The fix: quantile area and inverse-mass weights **9**
7.1 The quantile area . . . 9
7.2 Inverse-mass weights for perimeter and Euler . . . 10
7.3 Checks . . . 11

8 Where the flow-matching loss looks **11**

9 Is the tail too noisy to learn from? (“stability in the tails”) **13**
9.1 The first-order argument . . . 13
9.2 Beyond first order: synthetic and real tests . . . 14

10 The general hypothesis **14**

11 The deterministic U-Net with the quantile form **16**

12 Risks, open items and the plan **16**

1 The problem in one page

We want 2-km precipitation fields from 12.5-km inputs that get *heavy rain* right: how much of it there is, how intense it gets and what shape it has. Two observations drive the whole project.

1. **Pixel losses do not care about heavy rain.** Pixels above 20 mm are about 10^{-3} of the pixels of a heavy patch and 10^{-4} of a random one. Any loss that averages over pixels gives them a tiny share of its gradient, so a model trained on it can afford to get them wrong. Section 8 measures this for the flow-matching loss: about 1% of its gradient.
2. **Structural statistics see heavy rain directly.** The Minkowski functionals describe the set of pixels above a threshold u (its area, its perimeter, how many pieces it has) at many thresholds, including high ones. A loss on them can, in principle, give the top of the distribution its own weight.

The notes follow one question: *does the Minkowski loss actually put gradient on heavy rain, and if not, why not and how do we fix it?* The short answer, built up in the sections below:

- In its original form the loss gives each threshold a pull proportional to the *number of pixels near that threshold*, so light rain still dominates, and thresholds the prediction never reaches give *no* gradient at all (Section 6).
- The fix (`REWARD_FORM: quantile_mass`) reads the area curve along the intensity axis and reweights perimeter and Euler by inverse pixel mass. It moves the share of gradient on pixels ≥ 20 mm from 3.7% to 40.9% on heavy patches (Section 7).
- The tail mismatch is not hidden by noise: every reward feature sees it at 1–3 times the member spread. What blocked the original form was *reach* (Section 9).

2 Fields, the two spaces, and the base models

2.1 Model space

A precipitation field is a vector $f \in \mathbb{R}^N_{\geq 0}$ of hourly accumulations in mm, $N = 128^2 = 16{,}384$ pixels of 2×2 km. Networks work in the *model space*

$$z = \frac{\log(1+f)}{c}, \qquad f = e^{cz} - 1, \qquad c = \log(1 + 613 \text{ mm}) = 6.42, \tag{1}$$

so $z \in [0, 1]$. Two identities return repeatedly: $\log(1+f) = cz$, and $\partial f/\partial z = c\,(1+f)$. The second says that a small step in z is a large step in mm for a heavy pixel: at $f = 50$ mm, $\partial f/\partial z = 327$ mm.

2.2 The deterministic backbone and its loss (Study 1)

A U-Net maps the coarse input X to a prediction $\hat{z} = \mu_\theta(X)$. The vanilla loss is the pixel mean squared error in model space,

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{z}_i - z_i\right)^2. \tag{2}$$

What it converges to. [derivation] For one pixel, $\min_a \mathbb{E}[(a-Z)^2 \mid X]$ is reached at $a^\star = \mathbb{E}[Z \mid X]$, the conditional mean. Where the coarse input cannot tell where inside a 12.5-km cell the core will be, the conditional mean spreads the core over all the places it could be.

Worked example. *Suppose that, given X, a 2-km pixel has a 20% chance of 50 mm and an 80% chance of 1 mm. In model space, $z = \log(51)/6.42 = 0.612$ or $\log 2/6.42 = 0.108$. The MSE optimum is $a^\star = 0.2 \times 0.612 + 0.8 \times 0.108 = 0.209$, i.e. $f = e^{6.42 \times 0.209} - 1 = 2.8$ mm. The model predicts 2.8 mm everywhere and never 50 mm. The mean in log space is a geometric-type mean, which falls even further below the 50 mm peak than the arithmetic mean (10.8 mm) would.*

This is the smoothing that the structural loss is meant to counteract in Study 1.

The structural loss in Study 1. The deterministic objective is

$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{MSE}} + w(e)\, D_{\text{Mink}}\big(f(\hat{z}),\, g^{\text{obs}}\big), \tag{3}$$

with D_{Mink} the distance of Section 3 summed over its three channels, and $w(e)$ a cosine warm-up from 0 to $w_{\max}$ over the first epochs (`src/trainers/unet_analytical.py`). Hourly data needed a new $w_{\max}$ (DECISIONS §37); Section 3.1 summarises the bracket.

2.3 The flow-matching residual model (Study 2)

Flow matching (FM) learns the part of the target the backbone cannot predict. With μ the backbone prediction and Y the target in model space, the *residual* is

$$r = \frac{Y - \mu}{\sigma_r}, \qquad \sigma_r = 0.01267 \quad \text{(hourly vanilla backbone, train split)}, \tag{4}$$

scaled to be roughly unit size. FM learns a velocity field $v_\theta(x, t, \mathbf{c})$, condition $\mathbf{c} = [X, \mu]$, that carries Gaussian noise x_0 at $t = 0$ to residuals at $t = 1$. Training draws $x_0 \sim \mathcal{N}(0, I)$, $t \sim U(0, 1)$, forms the straight path $x_t = (1-t)x_0 + t\,r$ and regresses the velocity of that path:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, x_0, (r, \mathbf{c})}\, \frac{1}{N} \big\| v_\theta(x_t, t, \mathbf{c}) - (r - x_0) \big\|^2. \tag{5}$$

What it converges to, and why that differs from the backbone. [derivation] Again an MSE, so the optimum is a conditional mean, but now of the *path velocity given the current state*: $v^\star(x, t, \mathbf{c}) = \mathbb{E}[r - x_0 \mid x_t = x, \mathbf{c}]$. Integrating $\mathrm{d}x/\mathrm{d}t = v^\star$ from noise transports the noise distribution exactly onto $p(r \mid \mathbf{c})$ [CITE?: flow matching / rectified flow, marginal velocity theorem]. So the FM loss is *not* biased toward the mean in principle: at its optimum the sampler reproduces the whole conditional distribution, tail included. The bias toward the mean in the backbone comes from asking for one field; FM asks for a velocity at every intermediate state, which is a different question.

What goes wrong in practice. The optimum is not reached. With a finite network and finite training, the error ends up where the loss is least sensitive. Equation (5) is a pixel mean, so heavy pixels are a tiny part of it. Section 8 measures how tiny. The clean hourly FM produces about 3 in 10 of the observed exceedances of 50.1 mm on heavy patches (Table 2).

Sampling. Heun's method, 16 steps of $h = 1/16$:

$$x_{k+1} = x_k + \tfrac{h}{2}\Big[v_\theta(x_k, t_k) + v_\theta\big(x_k + h\, v_\theta(x_k, t_k),\, t_k + h\big) \Big], \tag{6}$$

and the generated field is $f = e^{c\, \text{clamp}(\mu + \sigma_r x_{16},\, 0,\, 1)} - 1$.

3 The Minkowski functionals and the structural distance

What is measured. For a threshold u, the *excursion set* is $E_u = \{i : f_i \geq u\}$. Three numbers describe its geometry: its area $A(u)$, its perimeter $P(u)$ and its Euler characteristic $\chi(u)$ (number of pieces minus number of holes). In two dimensions these are, up to linear combination, the only additive, continuous, motion-invariant functionals of a set [CITE?: Hadwiger's characterisation theorem]. The loss uses $Q = 15$ thresholds, evenly spaced in $\xi = \log u$:

$$u \in \{0.1,\ 0.192,\ 0.314,\ 0.509,\ 0.828,\ 1.35,\ 2.11,\ 3.43,\ 5.36,\ 8.61,\ 13.5,\ 20.2,\ 31.7,\ 50.1,\ 79.0\}\,\text{mm}.$$

Discrete, differentiable estimators (`src/losses/minkowski.py`). The hard indicator $\mathbb{1}[f_i \geq u]$ has zero derivative almost everywhere. The loss therefore uses a *soft mask*

$$s_i(u) = \sigma\left(\frac{f_i - u}{\tau_u}\right), \qquad \tau_u = \max(0.1\, a\, u,\ 10^{-3}), \tag{7}$$

with σ the logistic function and anneal factor a; at $a = 0.05$ (the reward), $\tau_u = 0.005\, u$, which is 0.25 mm at $u = 50$ mm. Then

$$A(u) = \ell^2 \sum_i \mathbb{1}[f_i \geq u] \ \text{(forward)}, \qquad \frac{\partial A}{\partial f_i} := \ell^2 \frac{\partial s_i}{\partial f_i} \ \text{(backward)}, \tag{8}$$

$$P(u) = \ell\Big[w_{\text{ax}} \sum_{\text{axial pairs}} |s_i - s_j| + w_{\text{di}} \sum_{\text{diagonal pairs}} |s_i - s_j|\Big], \tag{9}$$

$$\chi(u) = V - E_x - E_y + F, \tag{10}$$

with $\ell = 2$ km, Cauchy–Crofton weights $w_{\text{ax}} = 0.473$, $w_{\text{di}} = 0.218$, and V, E, F soft counts of pixels, edges and faces (an edge or face counts with the minimum of its pixels' masks). The area uses a *straight-through estimator*: the exact count in the forward pass, the soft-mask derivative in the backward pass.

The curves and the distance. Each field becomes three curves over the 15 thresholds,

$$g(f) = \big[\log(1 + A),\ \log(1 + P),\ \operatorname{sign}(\chi)\log(1 + |\chi|)\big] \in \mathbb{R}^{3\times 15}, \tag{11}$$

and two curves are compared by integrating the absolute difference over log-intensity,

$$d(g, h) = \frac{1}{3}\sum_{k=1}^{3} \frac{1}{\xi_{15} - \xi_1} \int_{\xi_1}^{\xi_{15}} w_k(\xi)\, \big|g_k(\xi) - h_k(\xi)\big|\, \mathrm{d}\xi, \tag{12}$$

evaluated by the trapezoid rule on the thresholds, with weights $w_k \equiv 1$ in the original form (Section 7 introduces other weights). The deterministic loss (3) uses the channel *sum*, $D_{\text{Mink}} = 3d$; the reward uses the channel mean d. Every decade of intensity gets equal weight *in this sum*. Section 6 shows why that is not equal weight in the gradient.

What the functionals cannot see. They do not change when the field is shifted or rotated. A loss on them cannot tell the model *where* rain falls; that must come from a pixel-aligned term (the MSE, or the FM loss). This is why no Minkowski variant improves FSS at the 1-pixel scale.

3.1 Study 1 on hourly data, in brief

[measured, DECISIONS §37] Bracket of $w_{\max}$, validation, last checkpoints, $u = 50.1$ mm:

$w_{\max}$	ΔMAE	exceedance ratio	RL bias λ10 / 100 / 1000	FSS 50.1 mm 1 / 21 px
0 (vanilla)	0	0.113	−44 / −31 / −13%	0.106 / 0.383
6.0×10^{-5}	+4.7%	0.192	−37 / −41 / −44%	0.097 / 0.417
1.8×10^{-4}	+7.9%	0.287	−32 / −36 / −41%	0.118 / 0.535
5.4×10^{-4}	+12.5%	0.442	−25 / −32 / −39%	0.131 / 0.648
1.8×10^{-3}	+19.0%	0.607	−18 / −21 / −24%	0.125 / 0.714

More weight gives more exceedances, higher peaks and better FSS at 21 px, at a rising cost in MAE and, visibly in the images, grid-aligned striping from 1.8×10^{-4} upward. Only 6×10^{-5} is inside the +5% MAE cap. Every Minkowski run under-predicts the long return levels more than vanilla does. The Study-1 weight on hourly data is still open. All these runs use the *original* threshold form, so everything in Section 6 applies to them too.

4 How the Minkowski loss is coupled to flow matching

Why it cannot simply be added. The FM loss (5) lives on noisy intermediate states x_t, which are not precipitation fields. The functionals only make sense on a finished field. The fine-tune therefore treats the structural distance as a *reward* on generated samples, following DRaFT-K [2]: generate a sample, score it, and back-propagate the score through the sampler into θ.

DRaFT-K with $K = 1$. Back-propagating through all 16 Heun steps would store 32 network evaluations and spread the credit thinly over the path. DRaFT-K cuts the graph: the first $16 - K$ steps run with gradients off. With $K = 1$,

$$x_{15} = \mathrm{sg}\big(15 \text{ Heun steps from } x_0\big), \qquad x_{16} = \Phi_\theta(x_{15}), \qquad \nabla_\theta R = \frac{\partial R}{\partial x_{16}} \, \frac{\partial \Phi_\theta(x_{15})}{\partial \theta}, \tag{13}$$

where sg stops the gradient and Φ_θ is the last Heun step (6). Only the last 1/16 of the path is trained by the reward. [inference] In FM the last step mostly sets fine detail and amplitude, so a $K = 1$ reward acts on the smallest scales. The $K = 1$ and $K = 2$ gradients are aligned (cosine +0.97, [measured], `logs/coupling_v4adv_*.log`), so $K = 1$ is not missing a direction that $K = 2$ would see.

One optimiser step. For a batch of $B = 8$ conditions, draw M members per condition, decode each to mm, compute g for each member, compare with the target curve g^{obs}. The total fine-tune loss is

$$L = w_{\mathrm{r}} \, R + w_{\mathrm{p}} \, \mathcal{L}_{\mathrm{prox}} + w_{\mathrm{ret}} \, \mathcal{L}_{\mathrm{FM}}, \tag{14}$$

optimised with AdamW, learning rate 10^{-5}, gradient clipping at norm 1, 10 epochs of 500 steps.

	FM + dist	FM + energy
reward R	distributional (15)	energy score (17)
members M	4	16
w_{r} / w_{p} / w_{ret}	10^{-3} / 1 / **0**	10^{-3} / 1 / **1**
spread weight λ_s	0.5	(not used)

Table 1: The two hourly fine-tunes (startup lines of `logs/finetune_v4adv_{dist,energy}.log`; both start from the clean FM, both use `config FM_RESIDUAL_SCALE=0.01267`).

Where the mean and the MSE meet. A common question is how the FM "MSE" and the Minkowski term interact. In the distributional run they are *not added*: $w_{\text{ret}} = 0$. The MSE acts in three indirect ways. (1) It produced the starting model. (2) The proximal anchor (below) is itself an MSE between the fine-tuned and the frozen velocity field, a limit on how far the model may move, not a data term. (3) In the energy run it is present as the retention term, and there, as Section 5 shows, it dominates.

4.1 The distributional reward

$$R_{\text{dist}} = \frac{1}{B}\sum_b d\Big(\tfrac{1}{M}\textstyle\sum_m g_{bm},\ g_b^{\text{obs}}\Big) + \lambda_s\, d\Big(\log\big(1+\text{std}_{b,m}\ g_{bm}\big),\ \log\big(1+\text{std}_b\ g_b^{\text{obs}}\big)\Big). \tag{15}$$

The first term asks the *average* member curve of each condition to match the observation. The second asks the spread of the curves to match the spread of the observed curves.

What the spread term actually measures. [derivation] Both standard deviations are pooled over the batch: $\text{std}_{b,m}$ runs over all $M \times B = 32$ generated curves. By the law of total variance,

$$\text{Var}_{b,m}(g) = \underbrace{\mathbb{E}_b\big[\,\text{Var}_m(g \mid b)\big]}_{\text{ensemble spread}} + \underbrace{\text{Var}_b\left(\mathbb{E}_m[g \mid b]\right)}_{\text{differences between patches}}. \tag{16}$$

Eight different patches differ far more than four members of one patch do, so the second term dominates. As written, the spread term mostly asks the eight patches to differ as much as their targets do. It does little to control the ensemble's own spread. R_{dist} is *not* a proper scoring rule (next subsection): a model can lower it without matching the true conditional distribution.

4.2 The energy reward

$$R_{\text{ES}} = \frac{1}{B}\sum_b \Big[\frac{1}{M}\sum_m d\big(g_{bm}, g_b^{\text{obs}}\big) - \frac{1}{2M(M-1)}\sum_{m\neq m'} d\big(g_{bm}, g_{bm'}\big)\Big]. \tag{17}$$

The first term pulls every member toward the observation; the second rewards members that differ from each other. This is an unbiased estimate of the *energy score* $\text{ES}(P, y) = \mathbb{E}_P\, d(G, y) - \frac{1}{2}\mathbb{E}_P\, d(G, G')$, with G, G' independent draws from the model [CITE?: Gneiting & Raftery 2007, energy score].

Why $M(M-1)$ and not M^2. [derivation] For independent members, $\mathbb{E}\big[\frac{1}{M(M-1)}\sum_{m\neq m'} d(G_m, G_{m'})\big] = \mathbb{E}\, d(G, G')$. Dividing the same sum by M^2 gives $\frac{M-1}{M}\mathbb{E}\, d(G, G')$: the bonus for spread shrinks, and the minimiser is an ensemble that is too narrow, which is the failure the reward is meant to correct.

Proper scores. A scoring rule is *proper* if its expected value is smallest when the model's distribution equals the true one. For a distance d of negative type, the energy score is proper: the gap between using P and the truth Q is half the energy distance between them, which is non-negative. The trapezoid integral (12) is a weighted L^1 distance between 45-dimensional vectors, and L^1 distances are of negative type, so R_{ES} is proper for the distribution of the curves g [derivation, given the negative-type property]. Whether it is *strictly* proper (the truth is the unique minimiser) with an L^1 distance, we do not know [CITE?: strong negative type of ℓ^1 metrics]. And proper only ever means proper *on the features*: two field distributions with the same distribution of curves score the same.

4.3 The proximal anchor and the retention term

$$\mathcal{L}_{\text{prox}} = \left\| v_\theta(\tilde{x}_t, t, \mathbf{c}) - v_{\text{ref}}(\tilde{x}_t, t, \mathbf{c}) \right\|^2, \qquad \tilde{x}_t = (1-t)\, x_0 + t\, \tilde{x}_1, \tag{18}$$

where v_{ref} is the frozen clean model. It should keep the fine-tuned velocity close to the clean one where the sampler goes. The retention term is the FM loss (5) on the true residual; it is the only pixel-aligned data term available, since the reward cannot see position.

The anchor is evaluated where the sampler never goes. [derivation from the code, `reward_finetune.py`, the `x1 = ...` line] The code sets $\tilde{x}_1 = \text{clamp}(f/6.42,\, 0,\, 1)$: the generated field *in mm*, divided by c and saturating at 6.4 mm. The model's coordinate is the scaled log-residual $r = (Y - \mu)/\sigma_r$, mostly near 0 and of order ± 1, with outliers to $|r| \approx 34$.

Worked example. *A pixel with 10 mm gives* $\tilde{x}_1 = \min(10/6.42, 1) = 1$ *wherever it is. A residual of* 0.05 *in model space (the backbone 0.05 below the target, a factor* $e^{0.05c} \approx 1.38$ *in* $1 + f$*) is* $r = 0.05/0.01267 \approx 3.9$.

So $\tilde{x}_t$ blends noise with a binary-looking rain mask, states the ODE does not visit, and the anchor constrains the velocity there. In the dist run, which has no retention term, this anchor is the only thing holding the model near the clean one. This must be fixed before any new fine-tune (Section 12).

5 How the terms share the update: the gradient budget

Weights are not what matters; gradient mix is. [derivation] Adam's update is $\Delta\theta = -\eta\, \hat{m}/(\sqrt{\hat{v}} + \epsilon)$, with $\hat{m}, \hat{v}$ running averages of the gradient and its square. Multiplying every gradient by a constant λ multiplies $\hat{m}$ by λ and $\sqrt{\hat{v}}$ by $|\lambda|$, so the step is unchanged (while ϵ is negligible). Two consequences:

- A loss whose total gradient is tiny (the dist run: norm 8×10^{-4}, far below the clip at 1) still makes full-size steps, all in its own direction.
- When terms are added, what decides the direction is the *ratio* of their gradients, not the weights. $w_{\text{r}} = 10^{-3}$ means nothing until it is compared with the other terms' gradient norms.

Measured at the clean checkpoint (`scripts/evaluate/grad_norm_terms.py`, `logs/grad_norm_terms_v4adv.txt`, four training batches) [measured]:

run	reward	anchor	FM retention	reward share	$\cos(\nabla R, \nabla L)$
energy, original form	8.3×10^{-4}	1.2×10^{-4}	0.65	0.13%	-0.01
dist, original form	8.3×10^{-4}	0.7×10^{-4}	0	99%	$+0.99$
energy, quantile_mass	5.0×10^{-4}		0.65	0.07%	-0.03
dist, quantile_mass	5.0×10^{-4}		0	96%	$+0.97$

Norms of ∇_θ of $w_{\text{r}} R$, $w_{\text{p}} \mathcal{L}_{\text{prox}}$, $w_{\text{ret}} \mathcal{L}_{\text{FM}}$ before clipping; share $= \|\nabla(w_{\text{r}} R)\| / \|\nabla L\|$.

In the energy configuration the reward is 0.1% of the gradient and orthogonal to the total. The energy fine-tune is therefore, to a good approximation, more FM training at a low learning rate. Matching the reward's gradient to the retention term's (the gradient-norm rule, DECISIONS §9) means raising w_{r} by about three orders of magnitude.

5.1 What the two fine-tunes did

[measured, DECISIONS §38] 16 members, heun-16, $u = 50.1$ mm, `fm_mink_latest.pth`. The clean FM was evaluated twice (the evaluation does not seed its noise), which gives a crude noise level.

heavy subset (1,024 patches)	FM draws 1 / 2	FM + dist	FM + energy
exceedances (of 1,537 observed)	458 / 393	**580**	411
RL bias λ10 / 100 / 1000	$-30/-36/-44$; $-33/-36/-36\%$	$\mathbf{-23}/\mathbf{-20}/\mathbf{-15}\%$	$-31/-33/-33\%$
FSS 31.7 mm, 21 px	0.707 / 0.707	**0.741**	0.726
FSS 50.1 mm, 1 / 21 px	0.114/0.506 ; 0.103/0.523	0.109 / **0.554**	0.086 / 0.469
RAPSD distance	1.14 / 1.19	**0.81**	1.24
anisotropy	1.31 / 1.27	1.30	1.28
CRPS	0.1613 / 0.1614	0.1598	0.1615
random subset (4,096): exceedance ratio	0.272	**0.517**	0.228
RAPSD distance	0.86	**0.35**	0.97

Table 2: Clean FM and its two fine-tunes on hourly data. One seed per model.

- **FM + dist** beats clean FM by more than the evaluation noise on the 100- and 1000-year return levels, FSS at 31.7 mm, the spectrum and CRPS; on the exceedance count by about 3 noise standard deviations. Yet its training reward barely moved (0.351 → 0.339), and its gradient was mostly on light rain. Why it helped the tail is not established; Section 8 offers a candidate.
- **FM + energy** sits inside the spread of the two clean-FM draws on every tail column and is slightly worse on FSS at 50.1 mm and on the spectrum. Its reward did not move (0.199 → 0.205). Given the budget table this is expected: the run tested nothing about the reward. Its one gain, FSS at 31.7 mm, cannot be separated from the 5,000 extra FM steps without the $w_{\mathrm{r}} = 0$ control. (Its GPD shape error is the best of the table, 0.011, next to a weak tail: shape and scale trade off, which is why the shape column is not used to rank.)

6 Why the original reward misses heavy rain

One pixel, one threshold. [derivation] Through the soft mask (7), pixel i feels threshold u only through

$$\frac{\partial s_i}{\partial f_i} = \frac{1}{\tau_u}\,\sigma'\!\left(\frac{f_i - u}{\tau_u}\right), \qquad \sigma'(a) = \sigma(a)\big(1-\sigma(a)\big) \approx e^{-|a|} \text{ for } |a| \gg 1. \tag{19}$$

The kernel has width $\tau_u = 0.005u$: a pixel feels only the thresholds within a few per cent of its own value.

Worked example. *A predicted core of 17 mm under a target that exceeds 20.2, 31.7 and 50.1 mm. For $u = 20.2$ mm, $\tau = 0.101$ mm and $a = (17-20.2)/0.101 = -31.7$, so $\partial s/\partial f \approx e^{-31.7}/0.101 \approx 1.7\times10^{-13}$ mm^{-1}. For $u = 50.1$, $a = -132$ and $e^{-132} \approx 5\times10^{-58}$, which underflows to zero in single precision. The thresholds where the error is largest are the ones that give no gradient. We call this the* ***reach*** *problem.*

All pixels, one threshold: the pixel-mass limit. [derivation] Summing (19) over pixels and writing p for the density of pixel values,

$$\sum_i \frac{\partial s_i}{\partial f_i} = N\int p(f)\,\frac{1}{\tau}\sigma'\!\Big(\frac{f-u}{\tau}\Big)\mathrm{d}f \;\longrightarrow\; N\,p(u) \quad (\tau \to 0), \tag{20}$$

because $\tau^{-1}\sigma'(\cdot/\tau)$ is a probability density of width $\sim\tau$ (it integrates to 1). A threshold's total pull is the number of pixels per mm at u, *whatever τ is.* Two results follow.

- Light rain has most pixels, so it takes most of the gradient: on the clean hourly FM, 73% of $|\partial R/\partial z|$ lands on pixels below 1 mm on the heavy subset, 89% on the random subset, and 3.7% / 0.2% on pixels ≥ 20 mm (Fig. 2) [measured].

- Changing the anneal factor changes τ, so it cannot change the shares to leading order. This explains the 15-min sweep (DECISIONS §5): anneal $0.05 \to 0.40$ raised the absolute tail gradient by eleven orders of magnitude while the tail's share stayed at 0.75–0.88%.

Misconception. "Equal weight per decade in the loss means equal weight per decade in training."

Where it fails. The distance (12) weights decades equally, but the gradient of each decade's term is multiplied by the number of pixels near it; above the prediction's maximum it is zero.

What it over-generalises. It is true for a quantity that is linear in the curve values. Here the curves' sensitivity to the field is itself proportional to pixel mass.

7 The fix: quantile area and inverse-mass weights

The two failures need different fixes: (1) the gradient grows with pixel mass; (2) a threshold out of reach gives nothing. Options considered, all keeping $K = 1$:

A. Inverse-mass weights. Weight each threshold's term by $(n_j + 1)^{-\alpha}$, n_j the target's number of pixels above u_j. Fixes (1), not (2).

B. Hard forward, heavy-tailed backward. Exact counts forward, a heavy-tailed kernel of width $\sim 0.3u$ backward. Fixes (2), but adds a width hyperparameter, and a surrogate gradient can point where the true objective does not, which is how gaming starts.

C. Quantile form of the area. Compare the area curve horizontally instead of vertically. Removes (2) exactly for the area, with no new hyperparameter.

Decision (researcher, 2026-10-29): C for the area, A for perimeter and Euler (`REWARD_FORM: quantile_mass`; the original stays as `REWARD_FORM: threshold`, the default).

7.1 The quantile area

Same curve, read along the other axis. The area curve is a *survival function*: $S_f(v) = \#\{i : f_i \geq v\}$, and $A(u) = \ell^2 S_f(u)$. Plot it with intensity horizontally and pixel count vertically (Fig. 1). The original loss compares the two curves *vertically*: at a fixed intensity u_j, how many pixels each field has above it. One can equally compare them *horizontally*: at a fixed count n, which intensity each field reaches. That intensity is the field's n-th largest value, $f_{(n)}$.

The new area channel. For condition b and threshold u_j, let $n_j = \#\{i : y_i \geq u_j\}$ be the target's count. The area entry becomes

$$q_j(f) = \log\big(1 + f_{(n_j)}\big), \qquad q_j(y) = \log\big(1 + y_{(n_j)}\big), \tag{21}$$

masked (weight 0) where $n_j = 0$, i.e. above the target maximum.

Its gradient. [derivation] In model space $\log(1 + f) = cz$, so $q_j(f) = c\, z_{(n_j)}$ and

$$\frac{\partial}{\partial z_i}\big|q_j(f) - q_j(y)\big| = \pm c \quad \text{for the pixel of rank } n_j, \quad 0 \text{ otherwise.} \tag{22}$$

Every populated threshold moves exactly one pixel per member, by the same amount c, whatever that pixel's value. (Sorting is differentiable in the sense needed: the sorted values are a permutation of the inputs, and the gradient flows to the selected element.) In Fig. 1 the term at $n = 7$ pushes the predicted 31.2 mm pixel up toward 53.7 mm, where the vertical term gives nothing.

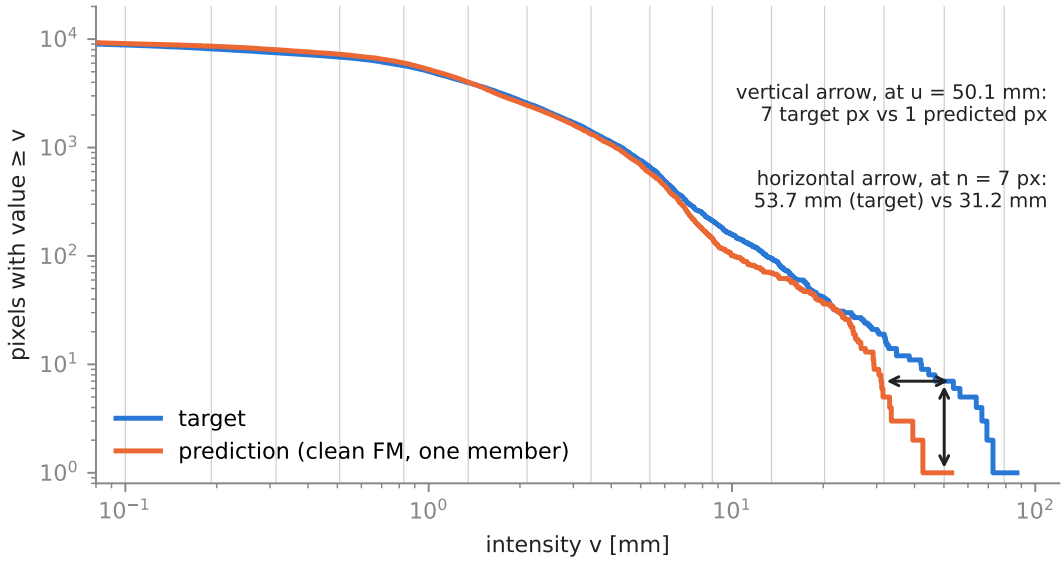


Figure 1: Survival curves of a heavy validation patch (patch 979, target maximum 87 mm) and one clean-FM member. Vertically, at $u = 50.1$ mm, the target has 7 pixels above the threshold and the prediction 1; the vertical term needs a predicted pixel near 50.1 mm to have a gradient. Horizontally, at the same count $n = 7$, the target reaches 53.7 mm and the prediction 31.2 mm; this difference has a gradient on the predicted pixel of rank 7 wherever its value is.

Link to Wasserstein distances, and where it breaks. For two sets of N values, the 1-D Wasserstein distance is $W_1 = \frac{1}{N}\sum_n |f_{(n)} - y_{(n)}|$: a horizontal comparison at every rank. The same argument is made for distributions in [1]: a horizontal (transport) distance gives a usable gradient where a vertical comparison saturates because the supports do not overlap. Our area term is a horizontal comparison at 15 ranks only, in log-intensity, per patch. The analogy breaks in two places: at most 15 pixels per member get a gradient, and the comparison is per patch, not between distributions of patches.

7.2 Inverse-mass weights for perimeter and Euler

Perimeter and Euler stay vertical (they have no simple horizontal form) and are weighted per threshold,

$$w_j = \frac{(n_j + 1)^{-\alpha}}{\frac{1}{15}\sum_{j'}(n_{j'} + 1)^{-\alpha}}, \qquad \alpha = 1, \tag{23}$$

so the weights average 1. Thresholds above the target maximum take the weight of the highest populated threshold, so overshoot is penalised as strongly as the top of the target, not infinitely.

Why $\alpha = 1$. [derivation under an assumption] The pull of threshold u is the number of pixels near it. If the survival curve is locally a power law, $S(v) \propto v^{-k}$, the pixels per unit log-intensity at u are $-\mathrm{d}S/\mathrm{d}\log v = k\,S(u) \propto n_j$, and weighting by $1/n_j$ cancels it. The assumption fails where the curve bends (the flat top below 1 mm in Fig. 1), so $\alpha \in \{0, 0.5, 1\}$ were also checked directly, and $\alpha = 1$ spread the gradient most evenly.

Properness is kept. [derivation] Ranks and weights depend only on the target. For each condition the reward compares fixed feature vectors with a fixed weighted L^1 distance, so the energy score stays proper on these features. If the weights depended on the prediction, this would fail.

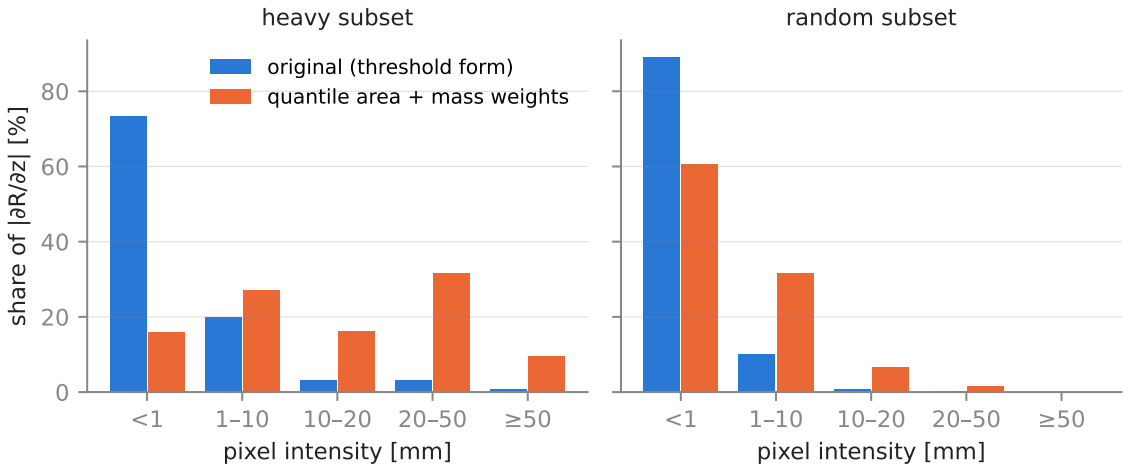


Figure 2: Share of the reward's model-space gradient $|\partial R/\partial z|$ on generated pixels of each intensity bin, original form vs quantile area + mass weights ($\alpha = 1$). Clean FM, energy mode, $M = 4$.

7.3 Checks

- **Unit tests** (`tests/test_reward_form.py`, 13 pass) [measured]: a 17 mm core under an 80 mm target gets no gradient from the original form and is pulled up by the new one, in all three modes; unit weights reproduce the original reward exactly; the rank feature, the masking and the weights behave as specified.
- **Where the gradient lands** (`scripts/evaluate/reward_threshold_share.py`) [measured], Fig. 2: on the heavy subset the share on pixels ≥ 20 mm goes from 3.7% to 40.9% and below 1 mm from 73% to 16%. With $\alpha = 0$ almost nothing changes: the area channel carried only 11% of the gradient, while the Euler term at 0.1 mm alone carried 17–28%.
- **Coupling checks** (`diagnose_coupling.py`, one heavy batch, fixed noise, 40 reward-only steps) [measured]: distributional -31.6% (original -28.1%), energy -35.1% (original -32.5%), both above their noise floors. The reward's sampling noise rises from 11.4% to 18.9% of its mean (distributional).

8 Where the flow-matching loss looks

The reward is meant to correct what the base loss neglects. To know how much it must correct, we measured how the FM loss itself spreads its gradient over intensity (`scripts/evaluate/fm_loss_intensity_share.py`, `logs/fm_loss_intensity_share_v4adv.txt`; heavy subset 1,024 patches $\times$ 4 noise draws, random subset 2,048 $\times$ 2) [measured].

How the split works. Equation (5) is a mean over pixels, so it splits exactly into one term per bin b of *target* intensity: $\mathcal{L}_{\mathrm{FM}} = \sum_b L_b$ with $L_b = \frac{1}{N}\sum_{i\in b}(v_i - u_i)^2$. For each bin we report

- its share of pixels, of the loss value, and of $\sum_{i\in b}|\partial L/\partial v_i|$, the gradient on the network's output (the analogue of $|\partial R/\partial z|$ for the reward);
- its *projection share* $\pi_b = \langle G_b, G\rangle/\|G\|^2$, with $G_b = \nabla_\theta L_b$ summed over all batches and $G = \sum_b G_b$.

Why the projection share: a gradient step $\Delta\theta = -\eta G$ changes bin b's loss, to first order, by $\Delta L_b \approx -\eta\,\langle G_b, G\rangle$. So π_b is the fraction of the first-order decrease of the loss that comes from bin

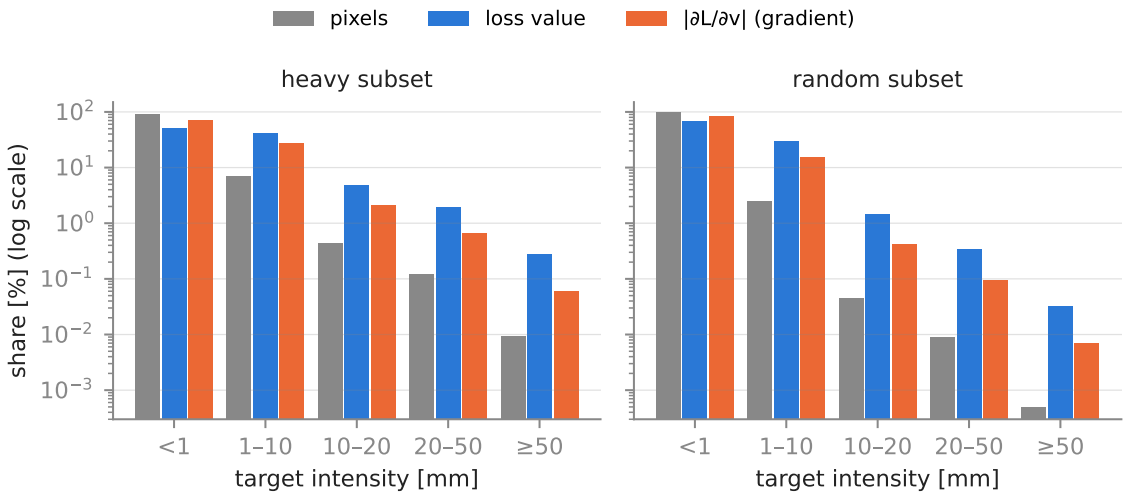


Figure 3: Share of pixels, of the FM loss value and of its output gradient $|\partial L/\partial v|$ per target-intensity bin, clean FM. Log scale: each grid line is a factor 10.

b. The π_b sum to 1; $\pi_b < 0$ means a step along the total gradient *increases* bin b's loss. (Adam rescales each parameter, so its step is not exactly $-\eta G$; π_b is the plain-gradient version.)

target mm	< 1	1–10	10–20	20–50	≥ 50
heavy: pixels %	92.3	7.2	0.44	0.12	0.009
heavy: $\lvert\partial L/\partial v\rvert$ %	69.9	27.3	2.1	0.66	0.06
heavy: projection share π_b	+327%	−170%	−37%	−17%	−2.5%
heavy: loss per pixel (/ mean)	0.56	5.8	11	16	30
random: pixels %	97.5	2.4	0.046	0.009	0.0005
random: $\lvert\partial L/\partial v\rvert$ %	84.3	15.2	0.43	0.09	0.007
random: projection share π_b	+65%	+38%	−1.9%	−1.5%	−0.2%

Three findings.

1. **Heavy rain gets about 1% of the gradient.** Pixels ≥ 20 mm carry 0.72% of $|\partial L/\partial v|$ on heavy patches and 0.10% on random ones.

2. **It is not simply proportional to pixel count.** The squared velocity error per pixel grows with intensity (16–30 times the mean above 20 mm on heavy patches), so the tail's share is 5–10 times its pixel share. The model *is* worse on heavy pixels, and the loss sees it, but not enough to outweigh their rarity. [withdrawn: "any pixel-averaged loss weights an intensity level by its pixel mass" (said in discussion on 2026-10-29). True for the sensitivity of a threshold term, not for a squared error whose size grows with intensity.]

3. **The tail pulls against the net direction.** Every bin ≥ 10 mm has $\pi_b < 0$: a plain gradient step lowers the loss on dry pixels and raises it on heavy ones. The per-bin gradients largely cancel (on heavy patches the dry bin's gradient alone is six times the total). Near a minimum some cancellation is expected by construction, since the total gradient is small; the informative part is the sign. Where bulk and tail conflict, the bulk wins.

Comparison with the reward. The original reward put 3.7% of its output gradient on pixels ≥ 20 mm, about five times the FM loss's 0.72%. The comparison is rough: the reward is binned on the generated field and in z, the FM loss on the target and in v. [inference, untested] This relative tilt toward the tail may be why the distributional fine-tune moved the tail (Table 2) even though most of its gradient was on light rain: relative to the base loss, it was still tail-heavy.

Why not just reweight the FM loss? (the obvious alternative). [derivation] Give each pixel a weight $w(y)$ depending on the target. The per-pixel optimum of $\mathbb{E}[w(Y)(a-Y)^2 \mid X]$ is

$$a^\star = \frac{\mathbb{E}[w(Y)\,Y \mid X]}{\mathbb{E}[w(Y) \mid X]}, \tag{24}$$

a *weighted* mean, no longer the conditional mean. For FM the same happens to the velocity: the sampler's output distribution is tilted toward $p(r \mid \mathbf{c})\,w$, so it would over-produce heavy rain by construction. Weighting *inside a proper score* (Section 7) only changes how errors are traded off; the truth stays the minimiser. This is the contrastive reason for using a feature-space reward instead of a weighted pixel loss. An inverse-mass-weighted FM loss is still the natural baseline the reward should beat.

9 Is the tail too noisy to learn from? ("stability in the tails")

The question. A whiteboard argument proposed that the reward features vary more smoothly in the tail than the raw intensities, so they give a more *stable* learning signal there. With F the cumulative distribution of pixel values, the inequality read $\left|\partial_F \log P\right| > \left|\partial_F \log \gamma_i\right|$ for each feature γ_i. For the area channel there are two ways to read the same tail, and the question becomes: which is more reliable, the *count* reading $\log(1 + \#\{f \geq u\})$ at a fixed threshold, or the *rank* reading $\log(1 + f_{(n)})$ at a fixed count?

9.1 The first-order argument

[derivation] Use the tail coordinate $s = -\log(1-F)$, i.e. log of one over the exceedance probability. For a generalised Pareto tail with shape ξ (observed $\xi = 0.155$ on the heavy subset), the intensity at level s satisfies $\log(v - u_0) \approx \xi s + \text{const}$, while the count above it is $n = Ne^{-s}$. So

$$\frac{\partial \log(v-u_0)}{\partial s} = \xi, \qquad \frac{\partial \log n}{\partial s} = -1. \tag{25}$$

Read literally, the whiteboard inequality *fails* for the area channel whenever $\xi < 1$: the intensity changes *less* than the count per unit of tail level.

Noise. An order statistic deep in the tail fluctuates by about $1/\sqrt{n}$ in s. Mapped through the slopes above, that is $\xi/\sqrt{n}$ in $\log v$ and $1/\sqrt{n}$ in $\log n$. The rank reading is about $1/\xi$ times less noisy in its own units. My first test, `tailstab.py`, confirmed this: independent GPD pixels, 4,000 replicate patches, ratio of spreads 0.24–0.34.

But signal scales the same way. [derivation] The two readings are the same survival curve read along two axes. A model that is wrong by Δs in tail level is wrong by $\xi\Delta s$ in the rank reading and Δs in the count reading. Signal and noise are multiplied by the same slope, so to first order the two readings have *the same signal-to-noise ratio.* [withdrawn: "the rank reading is about $1/\xi \approx 6$ times more stable" (2026-10-29). It compared noise without signal.]

Misconception. "A feature with less noise is a better training signal."

Where it fails. A change of variables shrinks noise and signal by the same factor. What matters is signal over noise, in the same units.

What it over-generalises. It is right when the signal is fixed and only the measurement is noisy, as in instrument design.

9.2 Beyond first order: synthetic and real tests

Synthetic (`scripts/evaluate/tail_reading_synthetic.py`, CPU): GPD pixels with $\xi = 0.155$, 400 conditions of 16 members each, fixed thresholds as in the reward, and a model whose intensities are scaled by $k = 0.7$ (an under-predicting model). Ratio bias/spread:

	independent pixels		4-px spatial correlation	
target count n	count	rank	count	rank
5	−2.4	−3.4	−3.5	−2.3
20	−3.4	−6.5	−2.0	−1.7
100	−6.8	−12.4	−1.2	−1.7
500	−13.8	−21.8	−1.9	−2.9

With a calibrated model ($k = 1$), both readings give a mean |mismatch|/spread of about 0.8, as expected, so neither is biased by itself. With independent pixels the rank reading wins by 1.4–1.9 times; the reason is nonlinear, not noise: counts collapse toward zero once the model falls short, and $\log(1 + \text{count})$ has a floor at zero. Spatial correlation makes both readings much noisier (a few correlated cells hold every extreme pixel) and the ranking flips at $n = 5$. Masking thresholds where the target count is zero, as `quantile_mass` does, also creates a selection bias at small n (a calibrated model looks −1.2 to −1.6 spreads low at $n = 5$).

Real FM ensemble (`scripts/evaluate/tail_reading_snr.py`, `logs/tail_reading_snr_v4adv.txt`; 16 members, 256 heavy and 512 random patches) [measured]. For every patch and threshold the target reaches, d is the member mean minus the target and s the member standard deviation; the figure shows $\overline{d}/\mathrm{rms}(s)$ for the four features actually used by the reward, and the share of members whose maximum stays below u.

1. **The tail deficit is visible in every feature.** Above about 5 mm the ensemble sits 1–3 spreads below the target, and the gap grows with u. Noise is not what hid the tail from the reward.

2. **The rank reading is only modestly better**: 1.1–1.5 times the count reading's bias/spread between 3 and 50 mm, about even at 79 mm. Perimeter and Euler track the count reading closely.

3. **Reach separates the two forms.** At 20 mm, 55% of members never reach the threshold; at 50 mm, 73% (heavy patches). In the threshold form these members get no gradient even though their mismatch is 2–3 spreads; in the rank form they do.

Conclusion of this section. "Stability in the tails" does not decide between the readings. The case for `quantile_mass` rests on *reach* and on the *share of gradient* the tail receives.

10 The general hypothesis

The results above suggest a statement that does not depend on the Minkowski functionals.

Hypothesis. A model trained on a pixel-averaged objective under-predicts heavy rain mainly because the objective gives it a tiny share of its gradient (here ∼1%) and trades it off against the bulk of the field, not mainly because the model cannot represent it. An auxiliary objective helps the tail if

(i) it puts a larger share of its gradient on high intensities than the base objective does;
(ii) it gives a gradient to samples that fall short of the observed tail (reach);

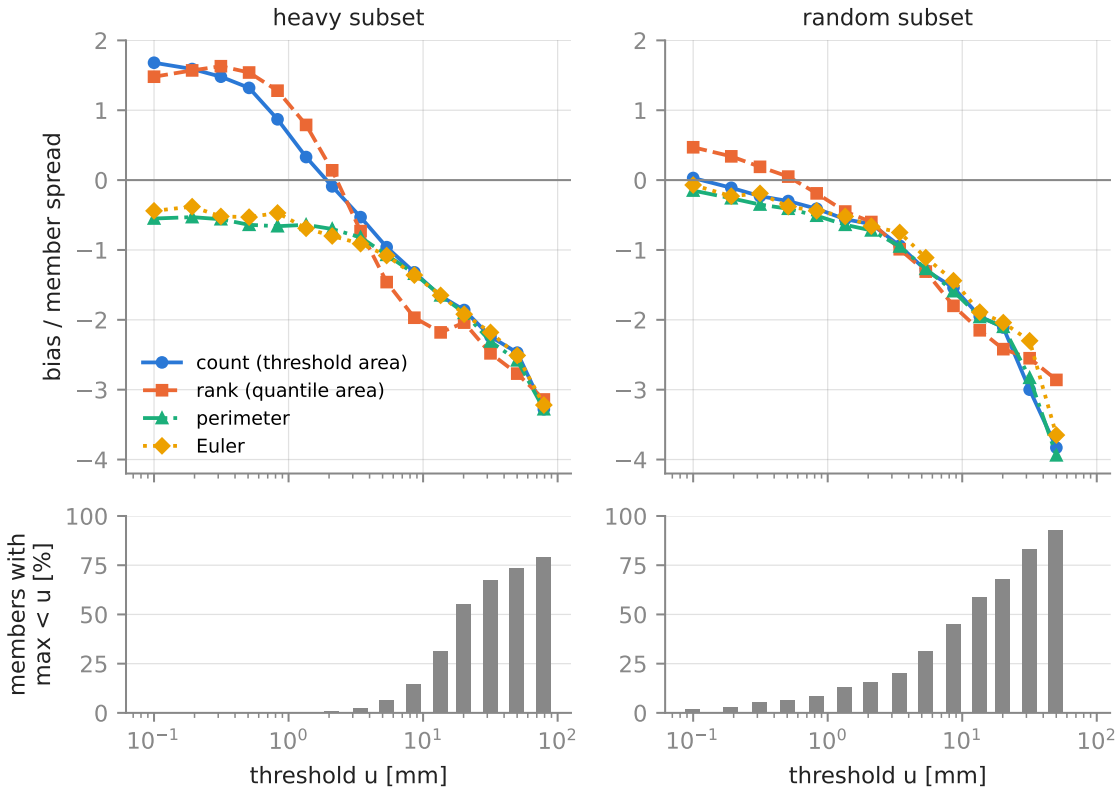


Figure 4: Top: systematic mismatch between the clean-FM ensemble and the target, in units of the ensemble spread, for the two area readings, perimeter and Euler; 0 is a calibrated model. Bottom: share of members that never reach the threshold.

(iii) it is a proper score on the quantities it compares, so it cannot be satisfied by a wrong distribution.

Why each condition is there. (i) is Section 8: the base loss leaves the tail almost unattended. (ii) is Sections 6 and 9: most samples never reach the high thresholds, and a vertical comparison is silent there. (iii) protects against the obvious way to satisfy (i) and (ii), overshooting: a reward that is not proper can be lowered by producing too much heavy rain, or the grid-aligned filaments seen at high Study-1 weights.

How to test it. Both (i) and (ii) are measurable for any candidate loss with the two new scripts: the share of gradient per intensity bin, and the share of members out of reach. The hypothesis predicts that, at matched total gradient norm, candidate losses rank on the tail columns as they rank on (i) and (ii). Candidates: threshold-weighted CRPS, a pinball loss at high quantile levels, a soft FSS at high thresholds, a GPD likelihood on exceedances, and the inverse-mass-weighted FM loss (which satisfies (i) and (ii) but not (iii)).

Where the hypothesis stops. Reweighting the intensity axis can fix *how much* heavy rain the model makes, not *where*. If the coarse input does not contain where the core will be, no weighting helps placement; FSS at 1 px barely changes in any run. The assumption that would most change the conclusion is that tail error comes from gradient allocation rather than from missing information in the input or limited capacity. The $w_r = 0$ control and the comparison across candidate losses test it.

The "η-learning" note. The whiteboard links "other reward?" to "η-learning". If η_j means per-threshold weights learned during training, then the $(n_j + 1)^{-1}$ weights are a fixed, hand-set η. Learning η by *minimising* the same loss collapses onto the easiest thresholds. It would need to be a min–max (the model minimises, η on a simplex maximises: robust over thresholds) or a constraint on the gradient share per intensity. [the meaning of the term is to be confirmed by the researcher]

11 The deterministic U-Net with the quantile form

The quantile form exists only in the FM reward (`reward_finetune.py:reward_features`). The Study-1 loss (3) still uses the threshold form, so every Study-1 run has the same pixel-mass and reach limits, and reach is probably worse there: the vanilla backbone's exceedance ratio is 0.05–0.11 against 0.27 for FM.

What porting it means. The deterministic loss has one prediction per condition, no ensemble. The quantile area becomes a per-sample term

$$D_{\text{area}} = \frac{1}{\xi_{15} - \xi_1} \int \mathbb{1}[n_j \geq 1] \left| \log(1 + \hat{f}_{(n_j)}) - \log(1 + y_{(n_j)}) \right| \mathrm{d}\xi,$$

a partial, log-scale 1-D Wasserstein distance between the predicted and observed distributions of pixel values in the patch, at 15 ranks. It needs the target *field*, not only its precomputed curves, in the loss call.

What to expect. [inference] For the backbone this term asks for the right *amount* of heavy rain per patch, wherever it is put, while the MSE penalises any core that is not in exactly the right place (the "double penalty"). The two terms pull against each other harder than in FM, where the reward and the FM loss act on different things. A new weight bracket is required: the form changes the loss's gradient norm, so the §37 weights do not carry over.

12 Risks, open items and the plan

Risks of the quantile form.

- **Noisier reward**: 11.4% → 18.9% of its mean (distributional). Training-time changes are harder to see on 20 validation batches.
- **Artefacts get more weight**: the top populated threshold may hold a handful of pixels, and the top of the distribution contains radar artefacts. Look at the images.
- **Sparse area gradient**: at most 15 pixels per member per step from the area term.
- **Only proper on the features**, and blind to position.
- **All gradient measurements are at the clean checkpoint**, before fine-tuning has moved the model.

Plan, in the researcher's order of priority (2026-10-30).

1. **FM + Minkowski reward (distributional), quantile form.**
   (a) Fix the proximal anchor so it is evaluated on the model's own coordinates (the scaled residual), with a unit test that the anchor is zero at initialisation and is evaluated on states the sampler visits.
   (b) Re-run the coupling check with the fixed anchor.
   (c) Fine-tune `config_v4_adv_fm_dist_qm.yaml`; check the startup lines (form, weights, `FM_RESIDUAL_SCALE` source); evaluate `fm_mink_latest.pth` on both subsets, against the clean FM draws and FM + dist (threshold form). Look at the images.

2. **FM + Minkowski reward (energy), quantile form.**
   (a) Rebalance w_{r} against w_{ret} by the gradient-norm rule (about $\times 1000$ on w_{r}), confirmed with `grad_norm_terms.py` before launching.
   (b) Run the $w_{\mathrm{r}} = 0$ control with the same configuration, seed and budget.
   (c) Fine-tune and evaluate as above.

3. **U-Net + Minkowski, quantile form.** Port `quantile_mass` to the deterministic loss with tests (a synthetic case with a known answer, and unit weights reproducing the current loss), measure its gradient share, then a new weight bracket.

Alongside: the fair trial of competing losses (roadmap M4) becomes the test of the general hypothesis, by measuring (i) and (ii) for each candidate before training it.

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN, 2017. URL https://arxiv.org/abs/1701.07875.

[2] Kevin Clark, Paul Vicol, Kevin Swersky, and David J. Fleet. Directly fine-tuning diffusion models on differentiable rewards. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17400.